

SOLID Principles — SRP & OCP (Session Notes)

The Problem: Design a Bird

Requirement

Design a `Bird` class with attributes and methods.

Version 0 — Initial Design

A single `Bird` class with:

- **Attributes:** weight, colour, type
- **Methods:** eat(), makeSound(), fly()

```
void makeSound() {  
    if (type == "sparrow") System.out.println("chi chi");  
    else if (type == "pigeon") System.out.println("gutargoon");  
    else if (type == "crow") System.out.println("kaw kaw");  
}
```

✗ Problems with Version 0

1. Readability

- Large number of bird types → long chain of `else-if` conditions
- Hard to read and maintain

2. Testability

- Each bird type needs a separate test case
- 500 birds → 500 test functions
- Becomes unscalable and impractical

3. Parallel Development

- Multiple developers modifying the same file
- Leads to frequent **git merge conflicts**
- Slows down development significantly

4. SRP Violation

`makeSound()` has **two reasons to change**:

- Adding a new bird type

- Modifying an existing bird's sound

➡ Multiple reasons to change = **SRP violation**

✓ Version 1 — Partial Fix (SRP Improvement)

```
void makeSound() {  
    if (type == "sparrow") makeSparrowSound();  
    if (type == "pigeon") makePigeonSound();  
}
```

Improvements

- Delegates logic into smaller functions
- Better readability
- `makeSound()` now has only **one reason to change** (new bird)

Still a Problem

- Adding new birds requires modifying existing code
 - ✗ Violates **OCP**
-

📌 SRP — Single Responsibility Principle

Definition: Every code unit (method, class, module) should have:

- One clear responsibility
- One reason to change

SRP Code Smells

- Long `else-if` or `switch` blocks
 - Monster methods (doing too many things)
 - Poorly organized utility/shared folders
-

📌 OCP — Open/Closed Principle

Definition: A class should be:

- **Closed for modification**
- **Open for extension**

Why?

- Prevents **regression bugs**
- Stable code is not modified → existing features remain safe

✓ Version 2 — SRP + OCP Solution

```
abstract class Bird {
    int weight;
    String colour;
    String type;

    abstract void eat();
    abstract void fly();
    abstract void makeSound();
}

class Sparrow extends Bird {
    void eat() {}
    void fly() {}
    void makeSound() {}
}

class Pigeon extends Bird {
    void eat() {}
    void fly() {}
    void makeSound() {}
}
```

Benefits

- New birds → new classes (no modification to existing code)
- Eliminates regression risk
- Each class has a single responsibility

➡ ✓ Best solution so far

📌 LSP — Liskov Substitution Principle (Intro)

Key Ideas

- Do not surprise the client
- Do not advertise behavior you cannot support
- Implementations must align with abstractions

⚠ Real-World Example (LSP Violation)

UberEats & Paytm Scenario

- A developer added a new `cancelled` status to a payment enum
- `checkout()` only checked: `status != failure`

- Cancelled payments were treated as successful

➡ Users received free food

Lesson

- Changing behavior unexpectedly breaks dependent systems
- Violates LSP

🔄 Versions 2.1 → 2.3 — Tackling LSP

Version 2.2

- Split into:
- FlyingBird
- NonFlyingBird

Version 2.3

- Further split into:
- flyingTalking
- flyingSilent
- nonFlyingTalking
- nonFlyingSilent

❌ Problem: Class Hierarchy Explosion

- With 2 features → 4 classes
- With 10 features → $2^{10} = 1024$ classes

➡ Not scalable ➡ Design becomes unmanageable

➡ Next Session

- Solve hierarchy explosion using a better design approach
- Likely introduction to **composition over inheritance / design patterns**

📊 UML & Design Diagrams

♦ Version 0 — Single Class (Problematic Design)

```
+-----+
|      Bird      |
+-----+
| weight         |
|               |
```

```

| colour          |
| type            |
+-----+
| eat()           |
| fly()           |
| makeSound()     |
+-----+

```

makeSound() → if-else chain based on type

◆ Version 1 — Delegation (Partial SRP Fix)

```

+-----+
|      Bird      |
+-----+
| type           |
+-----+
| makeSound()    |
| makeSparrowSound()
| makePigeonSound()
+-----+

```

➡ Logic separated, but still tightly coupled

◆ Version 2 — Proper OOP Design (SRP + OCP)

```

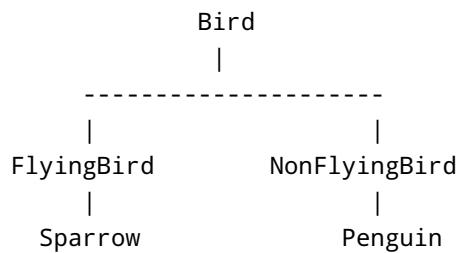
+-----+
| <<abstract>>   |
|      Bird      |
+-----+
| weight         |
| colour         |
+-----+
| eat()          |
| fly()          |
| makeSound()    |
+-----+
|
+-----+
|      |      |
+-----+      +-----+
| Sparrow |      | Pigeon |
+-----+      +-----+
| eat()   |      | eat()   |
| fly()   |      | fly()   |
+-----+      +-----+

```

makeSound()	makeSound()
+-----+	+-----+

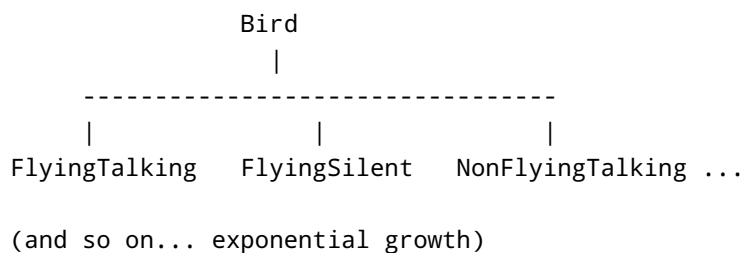
→ Each class handles its own behavior → No modification of existing code needed

♦ Version 2.2 — LSP Fix (Flying vs Non-Flying)



→ Prevents forcing all birds to implement fly()

♦ Version 2.3 — Hierarchy Explosion Problem



→ Adding features multiplies classes exponentially

↻ Evolution Flow

Version 0 → Version 1 → Version 2 → Version 2.2 → Version 2.3
 (if-else) (delegation) (OOP) (LSP fix) (problem)

→ Shows progression of design thinking

🧠 Quick Summary

- **Version 0:** Simple but violates SRP & OCP
- **Version 1:** Improves SRP, still violates OCP
- **Version 2:** Clean design using abstraction → follows SRP & OCP
- **LSP:** Ensures consistent behavior across hierarchy

- **Problem Ahead:** Too many subclasses → need better approach
-